Phase 1 Report: Cost-Aware Recovery for Multi-Agent Systems

Lucas Yan and Utkarsh Ranjan CSE 291A November 2026

Problem and Phase 1 Goal

Multi-agent language-model workflows can fail after completing useful work or causing external side effects. Restarting recomputes all prior work and cannot by itself undo irreversible effects. Our project asks: *given the known location of an injected fault, can provenance-guided recovery restore the task while recomputing less work than a full restart?* Fault detection and attribution are outside the current scope.

Phase 1 focused on making this question measurable. We built a deterministic travel-booking workflow with researcher, planner, and executor agents; a provenance graph that records steps, artifacts, dependencies, and effects; controlled fault injection; recovery baselines; and native adapters for external benchmarks. The implementation uses a fixed local Qwen2.5-7B-Instruct server for the planned main experiment, avoiding paid APIs.

Existing Solutions and Benchmark Choice

We reviewed recovery, failure-analysis, and adversarial-agent benchmarks. **UndoBench** provides state and side-effect oracles for operational tasks, including reversible, compensatable, and irreversible actions. **A2ASecBench** provides matched attack/control cases for agent-to-agent protocol threats. **TAMAS** covers malicious multi-agent instructions, but its released runner requires a paid model API, so we do not use it for Phase 1 numbers without course approval. **Who&When** labels decisive failure locations, but its released traces are not executable in our workflow and are therefore treated as attribution data rather than recovery trials.

The primary benchmark is our paired fault-injection harness because it directly supports the project assumption: the fault type and location are known, and every method starts from the same faulty execution. UndoBench adds an external side-effect oracle. A2ASecBench tests whether provenance can stop tainted downstream actions, but it measures prevention rather than post-effect rollback.

Implemented Evaluation

The seeded generator creates travel tasks with known flight, hotel, constraint, and email ground truth. The harness injects **10 fault types** across observations, agent outputs, plans, tool selection, and tool arguments. It implements full restart, suffix replay, coarse lineage, support lineage, monitor, support-monitor, and always-verify baselines.

We report five primary metrics: (1) task recovery success; (2) recovery side-effect integrity; (3) recovery-token ratio relative to restart; (4) recovery-latency ratio relative to restart; and (5) valid-work preservation. We additionally report final-effect integrity, irreversible-harm rate, recovery-introduced irreversible harm, and global/recovery unauthorized-action rates. This separation prevents an irreversible action from the faulty run, such as an already-sent email, from incorrectly turning successful task recovery into failure.

Evidence	Scale	Result	Interpretation
Automated tests	64	64 passed	Metrics, recovery, adapters, aggregation
Internal harness	10 faults	All fire in mock smoke test	Plumbing only; mock is not a result
UndoBench smoke	4 pairs	4/4 controls; 2/4 recovered	Recovery rate 0.50; duplicate-effect rate 0
A2ASecBench	4 suites, 800 records	Attack success 0; controls 1.0	One trial each; blocking, not rollback

Table 1: Quantitative Phase 1 evidence reproduced on November 3, 2026.

The four A2ASecBench suites were ASRF, ATSI, coordination oscillation, and high-output task flooding, with 200 records per suite. These perfect security scores are preliminary and policy-specific; HOTF uses a declared five-task fan-out limit and requires a threshold-sensitivity study. UndoBench currently retries only known pre-mutation failures once, so its 50% recovery rate is an honest capability boundary rather than a full-trajectory recovery claim.

Requirement Assessment and Remaining Work

The repository **meets the Phase 1 requirement**: it explores existing approaches, supplies a reproducible seeded dataset and fault suite, implements quantifiable evaluation metrics, integrates two executable external benchmarks, and passes 64 automated tests. The code also produces machine-readable rows and provenance traces for analysis.

It is **not yet the final empirical study**. Before the main cost-savings claim, we must run the fixed local model on 30 task seeds, 10 faults, and six recovery methods (1,800 paired comparisons); retain failures; repeat runs; report confidence intervals; use held-out seeds; sweep the HOTF quota; and exclude mock-LLM or unapproved paid-API results.

A Timeline and Milestones

Milestone	Deliverable and completion criterion
Phase 1: harness	**Complete.** Seeded workflow, provenance graph, 10 fault injections, effect classes, and deterministic oracle.
Phase 1: evaluation	**Complete.** Five primary metrics; separate final/recovery effects and irreversible harm; JSONL summaries and traces.
Phase 1: external validation	**Complete.** Native UndoBench and A2ASecBench adapters, 64 passing tests, and reproduced smoke results.
Phase 2, Week 1	Establish local-model clean-run reliability and determinism; freeze model, prompts, temperature, seeds, and fault schedules.
Phase 2, Week 2	Run 30 seeds $\times$ 10 faults $\times$ 6 recovery methods; retain failures and timeouts; compute paired token and latency ratios.
Phase 2, Week 3	Repeat runs, calculate confidence intervals, evaluate held-out seeds, and sweep HOTF task quotas (1, 5, 10, 20).
Final phase	Analyze failure cases and ablations, verify claims against artifacts, produce final tables/figures, and complete the report and presentation.

B Reproducibility Commands

```
uv sync
uv run pytest -q
./scripts/start_server.sh
uv run agentrec experiment --tasks 30 --faults all \
  --methods restart,suffix,coarse,support,monitor,support_monitor
uv run python analysis/summarize.py results/experiment-server-*/rows.jsonl
uv run agentrec undobench download
uv run agentrec undobench trial --suite smoke --recovery agentrec
uv run agentrec a2a download
uv run agentrec a2a trial --config configs/offline/asrf_eval.yaml --agentrec
```